Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Российский государственный университет им. А. Н. Косыгина (Технологии. Дизайн. Искусство)»
Институт информационных технологий и цифровой трансформации
Кафедра информационных технологий

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА К ЭСКИЗНОМУ ПРОЕКТУ

по лабораторной работе № 3

«Разработка эскизного проекта»

по дисциплине «Основы автоматизированного проектирования»

Программный продукт: «TransLark — интеллектуальный сервис для перевода и изучения иностранных языков»

Выполнил: студент группы ИТС-123
_____ (подпись)

Проверил: преподаватель
_____ (подпись)

Москва 2026

СОДЕРЖАНИЕ

1 Введение 2
2 Назначение и область применения 2
3 Технические характеристики 3
3.1 Постановка задачи 3
3.2 Архитектура программы 3
3.3 Описание алгоритмов 4
3.4 Организация входных и выходных данных 6
3.5 Выбор состава технических и программных средств 6
4 Ожидаемые технико-экономические показатели 7
5 Источники, использованные при разработке 7

1 Введение

Наименование программы — «TransLark», интеллектуальный сервис для перевода и изучения иностранных языков. Документ, на основании которого ведётся разработка, — техническое задание (ЛР 2). Эскизный проект выполнен по ГОСТ 19.105-78 и ГОСТ 19.404-79; термины — по ГОСТ 19781-90, схемы алгоритмов — по ГОСТ 19.701-90.

2 Назначение и область применения

Программа предназначена для перевода текстов, речи и изображений между 247 языками и для запоминания иностранной лексики методом интервальных повторений. Область применения — самостоятельное изучение языков, учебный процесс, поездки и работа с иноязычными материалами.

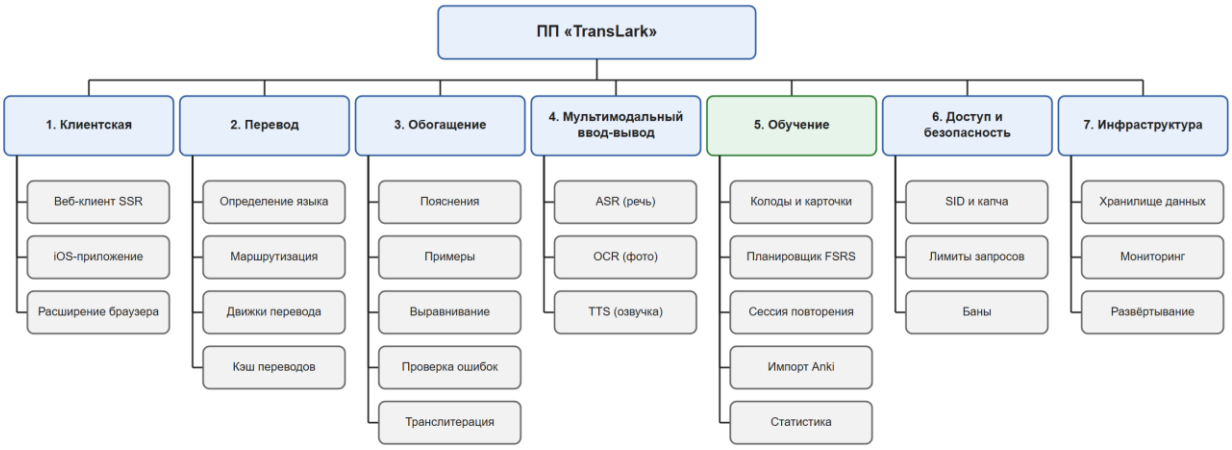
3 Технические характеристики

3.1 Постановка задачи

Требуется разработать программный комплекс, который по запросу пользователя: определяет язык текста; строит маршрут перевода между языками с наибольшим сквозным качеством; выполняет перевод одной из доступных моделей с резервированием; дополняет перевод пояснениями и примерами; создаёт учебную карточку и планирует её повторения так, чтобы вероятность вспомнить слово к моменту повтора была близка к заданной (0,9).

3.2 Архитектура программы

Архитектура описана на трёх уровнях иерархии (рисунок 3.1). Уровень 0 — программный продукт в целом. Уровень 1 — семь подсистем, полученных декомпозицией функций из ТЗ. Уровень 2 — модули каждой подсистемы.



Уровень 0 — продукт; уровень 1 — подсистемы; уровень 2 — модули

Рисунок 3.1 – Иерархия подсистем и модулей

Таблица 3.1 – Подсистемы и модули

Подсистема	Модули	Функции
1. Клиентская	веб-клиент SSR; iOS-приложение; расширение	ввод текста, отображение результата, локализация, обучение на устройстве
2. Перевод	detection; routing; translation (engines); cache	определение языка, граф качества пар, перевод по хопам, двухуровневый кэш
3. Обогащение	explain; examples; alignment; correct; transliterate	пояснения, примеры, соответствия, вычитка, транскрипция

Подсистема	Модули	Функции
4. Мультимодальный ввод-вывод	asr; ocr; tts	Whisper, PaddleOCR, синтез Silero/Piper/Kokoro
5. Обучение	decks; cards; scheduler; session; import; stats	колоды, карточки, FSRS, сессии, импорт .apkg, статистика
6. Доступ и безопасность	sid; captcha; rate-limit; bans	токены, капча, лимиты, блокировки
7. Инфраструктура	db; storage; metrics; deploy	PostgreSQL, MinIO, Prometheus, Docker Compose

Связи подсистем: клиентская подсистема обращается к подсистемам 2–5 через программный интерфейс REST; каждый запрос проходит через подсистему 6; подсистемы 2–5 используют общий кэш и хранилища подсистемы 7.

3.3 Описание алгоритмов

Алгоритм перевода (рисунок 3.2). Если язык источника не задан, сначала определяется язык. Затем проверяется кэш по ключу «пара языков + тон + SHA-256 нормализованного текста». При промахе строится маршрут: граф, где вершины — языки, а вес ребра равен $-\ln(q)$, где q — качество основной модели для пары. Кратчайший путь по алгоритму Флойда — Уоршелла даёт максимальное произведение качеств. Длина маршрута ограничена тремя переходами. Для каждого перехода движки перебираются в порядке приоритета: сильная LLM, локальная модель MADLAD-400, словарь. Если ответил не основной движок, результат помечается признаком degraded и кэшируется на 1 час вместо 90 дней.

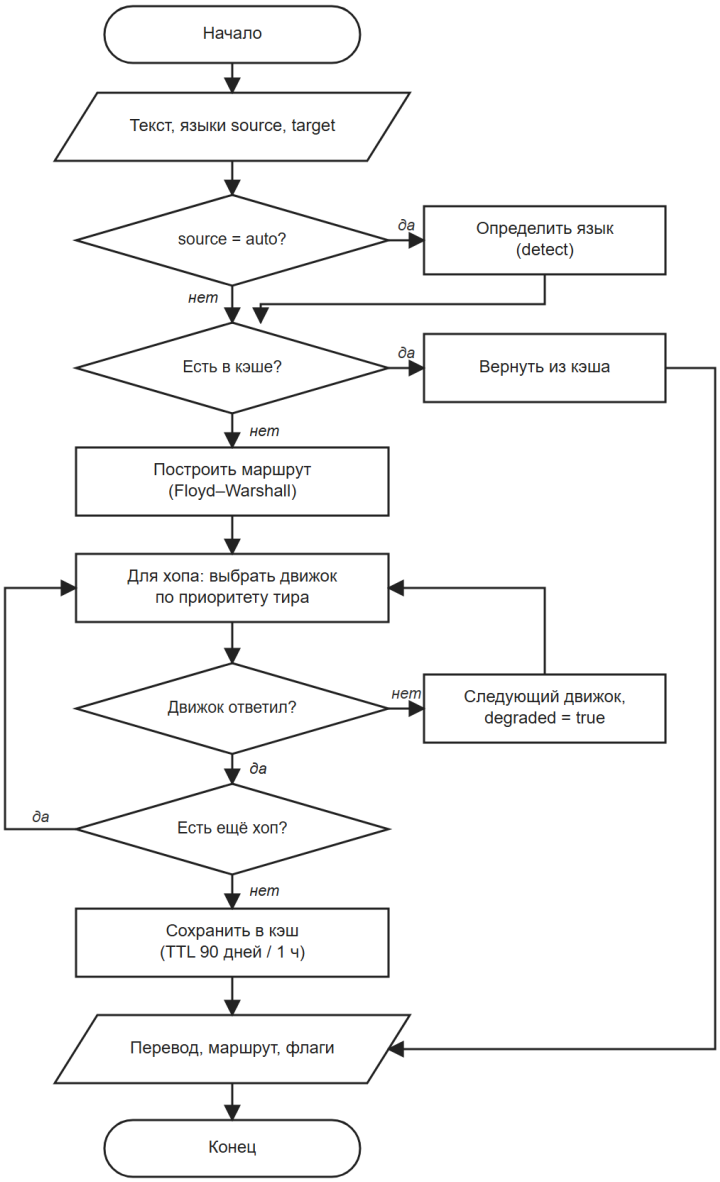


Рисунок 3.2 – Схема алгоритма перевода (ГОСТ 19.701-90)

Алгоритм повторения (рисунок 3.3). В очередь сессии отбираются карточки, срок повторения которых наступил. После ответа пользователь ставит оценку от 1 до 4. Планировщик FSRS по текущим стабильности S и трудности D и оценке вычисляет новые значения; интервал до следующего повтора выбирается так, чтобы извлекаемость R(t) = (1 + t/(9S))^−1 к этому моменту снизилась до целевых 0,9. Каждый ответ записывается в журнал повторений.

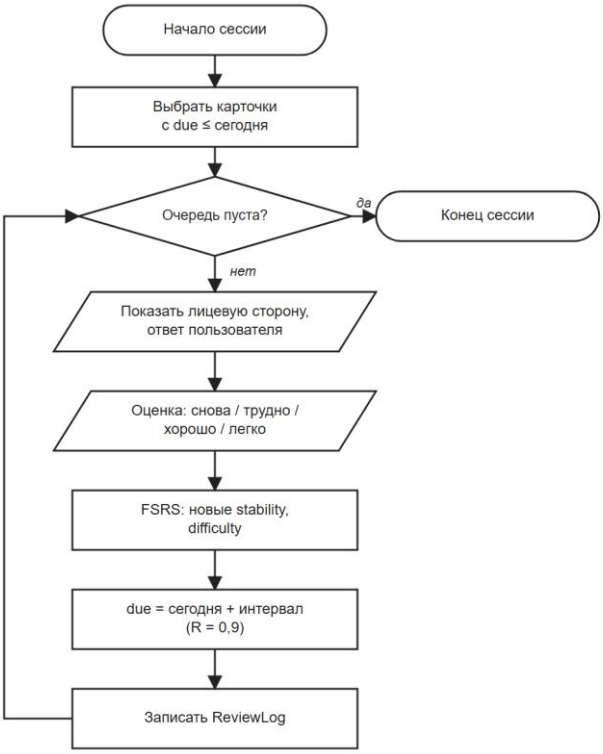


Рисунок 3.3 – Схема алгоритма сессии повторения

3.4 Организация входных и выходных данных

Таблица 3.2 – Входные и выходные данные

Функция	Вход	Выход
Перевод	JSON: text, source, target, tone	JSON: text, route, engine, degraded, needsExplanation, cached
Определение языка	JSON: text	JSON: language, confidence
Пояснение	JSON: text, translation, source, target	JSON: explanation
Распознавание речи	JSON: audio (base64), lang	JSON: text
Повторение	карточка, оценка 1–4, прошедшие дни	новое состояние памяти, дата повтора

3.5 Выбор состава технических и программных средств

Сервер: 8 ядер, 32 ГБ ОЗУ (модель MADLAD-400 3B в квантизации int8 занимает около 3 ГБ, Whisper — около 1,5 ГБ), SSD 200 ГБ, ОС Linux. Программные средства: Node.js 22 и NestJS 11 — единый язык TypeScript на клиенте и сервере; React 19 с серверным рендерингом — индексация поисковыми системами;

PostgreSQL 16 — надёжное хранение кэша и карточек; Python 3.11 с CTranslate2, faster-whisper, PaddleOCR — локальные модели без GPU; Docker Compose — воспроизводимое развёртывание.

4 Ожидаемые технико-экономические показатели

– время создания карточки сокращается с 1–2 минут при ручном переносе до 5 секунд;

– кэширование снижает число обращений к платным LLM на повторяющиеся запросы и укорачивает ответ до десятков миллисекунд;

– резервная локальная модель обеспечивает перевод при недоступности внешних API;

– эксплуатационные затраты ограничены арендой одного сервера и оплатой обращений к LLM.

5 Источники, использованные при разработке

1) ГОСТ 19781-90. Обеспечение систем обработки информации программное. Термины и определения.

2) ГОСТ 19.701-90. ЕСПД. Схемы алгоритмов, программ, данных и систем. Обозначения условные и правила выполнения.

3) ГОСТ 19.105-78. ЕСПД. Общие требования к программным документам.

4) ГОСТ 19.404-79. ЕСПД. Пояснительная записка. Требования к содержанию и оформлению.

5) Техническое задание на АС «TransLark» (ЛР 2).

6) Ye J. A Stochastic Shortest Path Algorithm for Optimizing Spaced Repetition Scheduling // Proc. KDD 2022. P. 4381–4390.